RIO: Runtime Intelligence Orchestration

A Technical White Paper on Governed AI Execution

Author: Brian K. Rasmussen **Version:** 1.0.0 **Date:** March 2026 **Repository:** github.com/bkr1297-RIO/rio-system

---

1. Abstract

Runtime Intelligence Orchestration (RIO) is a fail-closed authorization and audit runtime that governs the execution of actions performed by autonomous AI agents. RIO decouples the intelligence of an AI system from the authority to execute real-world actions by inserting a structural governance layer between the agent and the tools it controls. Every action follows a deterministic pipeline: intent is created, policy is evaluated, human approval is obtained when required, execution is gated, outcomes are verified, and a cryptographically signed receipt is appended to a tamper-evident ledger. The system enforces governance structurally at runtime rather than relying on model alignment, prompt engineering, or advisory guardrails.

---

2. Problem

Large Language Models have evolved from conversational interfaces into autonomous agents capable of sending emails, transferring funds, modifying infrastructure, and accessing sensitive data through API connectors and tool-use frameworks. This transition introduces a fundamental asymmetry: AI can propose and attempt actions at machine speed, while human oversight operates at human speed.

Current approaches to controlling AI agent behavior fall into three categories, all of which are insufficient for production environments:

Approach	Mechanism	Failure Mode
Prompt Engineering	System instructions telling the AI to “be careful”	Advisory only; overridden by prompt injection or model drift
Model Alignment	Training-time reinforcement of desired behaviors	Probabilistic; no runtime guarantee of compliance
UI Confirmation	Frontend dialog asking user to click “Confirm”	Client-side only; removable by any party with platform access

None of these approaches provide structural enforcement. They rely on the AI’s willingness to follow rules, the stability of trained behaviors, or the integrity of a client-side interface. In a production environment where an AI agent holds OAuth tokens to Gmail, GitHub, or a payment processor, the absence of a server-side execution gate means that a single prompt injection, a platform configuration change, or a model update can result in unauthorized actions with no audit trail.

The core problem is architectural: the tools that give AI agents the ability to act in the real world have no built-in governance layer. The connection between an AI agent and an external API is a direct pipe with no intermediary enforcing authorization, logging, or verification.

---

3. Solution

RIO addresses this gap by introducing a governed execution runtime that sits between AI agents and the external systems they interact with. RIO is not an AI model, not a chatbot framework, and not a monitoring dashboard. It is a control plane through which every AI-initiated action must pass before reaching the target system.

The runtime enforces three guarantees:

1. **No execution without authorization.** The system defaults to blocking all actions. Execution proceeds only after the governance pipeline produces a valid, signed approval. This is a fail-closed architecture: if any component is unavailable, missing, or invalid, execution is denied.

2. **Every action produces a cryptographic receipt.** Each completed action generates a receipt containing hashes of the original intent, the executed action,

and the verified outcome. Receipts are signed and appended to a hash-chained ledger, creating a tamper-evident audit trail.

3. **Governance is structural, not advisory.** Authorization is enforced server-side at the execution gateway. The AI agent cannot bypass the gate by modifying its own instructions, exploiting a prompt injection, or accessing tools directly. The gateway holds the OAuth tokens and API credentials; the agent does not.

---

4. Core Loop

RIO operates on a single deterministic pipeline that every action must traverse. The pipeline consists of four stages:

Intent → Governance → Execution → Receipt

4.1 Intent

The AI agent or an upstream system submits a structured intent describing the proposed action. The intent includes the action type (e.g., `send_email`, `create_file`, `transfer_funds`), the target system, the parameters, and the requesting identity. Once submitted, the intent is immutable and assigned a unique identifier.

4.2 Governance

The governance stage evaluates the intent against the active policy set. The policy engine classifies the action by risk level and returns one of three verdicts:

- **AUTO_APPROVE** — The action falls within pre-authorized boundaries (e.g., low-risk, routine operations). Execution proceeds without human intervention.
- **REQUIRE_HUMAN** — The action exceeds autonomous thresholds. Execution is blocked until a human authority reviews and approves the intent.
- **AUTO_DENY** — The action violates a hard policy constraint. Execution is permanently blocked.

For actions requiring human approval, the system presents the intent to the designated authority through the Gate UI. The authority reviews the proposed action, the AI's recommendation, and the risk assessment, then issues an approval or denial.

Approvals are cryptographically signed, time-limited, and bound to the specific intent identifier.

4.3 Execution

The execution gateway receives the approved intent and performs four verification checks before allowing the action to proceed:

1. A valid approval record exists and its cryptographic signature is verified.
2. The approval has not expired (default TTL: 300 seconds).
3. The approval has not been previously consumed (single-use enforcement).
4. The execution parameters match the original intent exactly (no parameter tampering).

If all checks pass, the gateway dispatches the action to the appropriate connector (Gmail, Google Drive, GitHub, etc.) and records the execution result. If any check fails, execution is denied and a denial receipt is generated.

4.4 Receipt

Immediately following execution, the system generates a cryptographic receipt that binds together the intent, the approval, and the execution outcome. The receipt is hashed, signed, and appended to the ledger. The receipt serves as non-repudiable proof that the action was authorized, executed as specified, and recorded.

---

5. The Four Invariants

RIO enforces four properties that every governed action must satisfy. These invariants are not configurable and cannot be overridden by policy.

5.1 Identity

Every action must be attributable to a specific requesting agent and a specific approving authority. The system records who requested the action (`requested_by`), who approved it (`approver_id`), and what system executed it (`executed_by`). Anonymous or unattributed actions are structurally impossible.

5.2 Consent

No action may execute without explicit authorization. For actions classified as `REQUIRE_HUMAN`, consent takes the form of a cryptographically signed approval record issued by a human authority. For actions classified as `AUTO_APPROVE`, consent is granted by the policy engine on behalf of the human who authored the policy. In both cases, consent is traceable to a human decision — either made in real time or codified in advance.

5.3 Persistence

Every action, whether approved, denied, or failed, produces an immutable record. Receipts are appended to a hash-chained ledger where each entry references the hash of the previous entry. Records cannot be modified or deleted after creation. If a record is later found to be erroneous, a compensating record is appended; the original is never altered.

5.4 Confidence

Every AI recommendation includes a confidence classification indicating the system's certainty about the proposed action. Confidence is expressed as a percentage tag (e.g., `[CONFIDENCE: 85%]`) accompanied by explicit declarations of assumptions and unknowns. Actions where confidence falls below a configurable threshold are automatically escalated to human review, regardless of the policy engine's initial verdict. Confidence ensures that the system does not proceed on uncertain ground without human judgment.

---

6. System Architecture

RIO consists of six core components. Each component has a defined responsibility and communicates with adjacent components through structured interfaces.

6.1 Component Overview

Component	Responsibility
Intent Service	Receives and records proposed actions as immutable intent objects
Policy Engine	Evaluates intents against the active policy set; returns AUTO_APPROVE, REQUIRE_HUMAN, or AUTO_DENY
Gate UI	Presents pending intents to human authorities for review and approval
Execution Gateway	Verifies approvals, enforces single-use tokens, dispatches actions to connectors
Connectors	Execute actions against external systems (Gmail, Drive, GitHub, Calendar)
Ledger	Stores signed receipts in a hash-chained, append-only audit log

6.2 Intent Service

The Intent Service accepts action proposals from AI agents or upstream systems. Each intent is assigned a UUID, timestamped, and stored as an immutable record. The intent includes the action type, target parameters, requesting identity, and risk metadata. Once created, an intent cannot be modified — any change requires a new intent submission.

6.3 Policy Engine

The Policy Engine evaluates each intent against a set of human-authored rules. Policies map action types, parameter ranges, and risk levels to governance verdicts. The engine uses a four-component risk scoring model:

- **Base Risk** — Inherent risk of the action type (e.g., `send_email` = medium, `transfer_funds` = high)
- **Role Modifier** — Adjusts based on the requesting agent's role and permissions
- **Amount Modifier** — Scales based on financial value or data volume involved
- **Target Modifier** — Adjusts based on the sensitivity of the target system

Policies follow a versioned lifecycle: `PROPOSED → APPROVED → ACTIVATED → INACTIVE`. Only one policy version may be active at any time, ensuring deterministic

evaluation.

6.4 Gate UI

The Gate UI is the human-facing interface where authorities review pending actions. It displays the original intent, the AI's recommendation (including confidence score and reasoning), and the policy engine's risk assessment. The authority can approve, deny, or request clarification. Approvals generate a signed approval record with a configurable time-to-live (default: 300 seconds).

6.5 Execution Gateway

The Execution Gateway is the enforcement point. It sits between the governance pipeline and the connectors. The gateway holds the OAuth tokens and API credentials for all connected external systems. The AI agent never has direct access to these credentials.

Before dispatching an action, the gateway verifies:

1. Approval record exists and signature is valid (ECDSA/Ed25519)
2. Approval has not expired
3. Approval has not been previously consumed (nonce registry)
4. Execution parameters match the original intent

If verification fails, the gateway returns HTTP 403 and generates a denial receipt. The gateway is fail-closed: if the approval service, ledger, or any verification component is unavailable, execution is blocked.

6.6 Connectors

Connectors are execution modules that interface with external systems. Each connector implements a standard interface:

Method	Purpose
`info()`	Returns connector metadata: platform, supported actions, connection status
`execute(request)`	Executes the authorized action against the target system
`verify(executionId)`	Independently verifies that the action was executed as authorized

Current registered connectors:

Connector	Platform	Actions	Status
Gmail	Google	`send_email`	Connected (live)
Google Drive	Google	`write_file`, `create_folder`, `share_file`	Connected (live)
GitHub	GitHub	`create_issue`, `create_pr`, `commit_file`	Connected (live)
Google Calendar	Google	`create_event`, `update_event`, `delete_event`	Simulated

Connectors only receive execution requests after the full governance pipeline has completed. No connector can be invoked directly by the AI agent.

6.7 Ledger

The Ledger is an append-only, hash-chained audit log. Every receipt appended to the ledger includes a `previous_hash` field containing the `ledger_hash` of the immediately preceding receipt. The first receipt in the chain uses the sentinel value `"GENESIS"`. This creates a tamper-evident chain: if any receipt is altered after the fact, its hash changes, breaking the link to all subsequent receipts and exposing the tampering.

Ledger verification follows a deterministic procedure:

1. Retrieve the most recent receipt.
2. Recompute the SHA-256 hash of the canonicalized receipt JSON.

3. Verify the recomputed hash matches the stored `ledger_hash`.
4. Verify the cryptographic signature using the issuing system's public key.
5. Retrieve the preceding receipt using `previous_hash`.
6. Repeat until the `GENESIS` receipt is reached.

If all hashes match and all signatures verify, the ledger is mathematically proven to be untampered.

---

7. Receipt Protocol

The receipt is the atomic unit of accountability in RIO. It is the cryptographic proof that an action was requested, authorized, executed, and recorded.

7.1 Receipt Structure

A receipt binds together the identifiers and hashes from every stage of the governance pipeline:

```
{
  "receipt_id": "RIO-1774769587",
  "request_id": "uuid-of-action-request",
  "recommendation_id": "uuid-of-ai-recommendation",
  "approval_id": "uuid-of-approval-record",
  "execution_id": "uuid-of-execution-record",
  "action_type": "send_email",
  "requested_by": "ai-agent-v1",
  "approver_id": "human-authority@example.com",
  "executed_by": "rio-executor-v0.1",
  "created_at": "2026-03-29T07:33:07.378704+00:00",
  "ledger_hash": "sha256-of-canonical-receipt",
  "previous_hash": "sha256-of-preceding-receipt",
  "chain_index": 0,
  "signature": "ecdsa-signature-over-canonical-form",
  "verification_method": "ecdsa_secp256k1",
  "status": "valid"
}
```

7.2 Hash Computation

Three hashes are computed for each governed action:

Hash	Input	Purpose
`intent_hash`	SHA-256 of the canonicalized intent JSON	Proves the original request has not been altered
`action_hash`	SHA-256 of the canonicalized execution parameters	Proves the executed action matches what was authorized
`verification_hash`	SHA-256 of the canonicalized execution result	Proves the outcome was recorded accurately

Canonicalization follows deterministic JSON serialization: keys are sorted alphabetically, no whitespace, UTF-8 encoding.

7.3 Signature

The receipt is signed using the issuing system's private key. Supported signature methods include `hmac_sha256`, `ecdsa_secp256k1`, `rsa_pss_sha256`, and `ed25519`. The signature covers the canonical form of the receipt including the `ledger_hash` but excluding the `signature` field itself.

7.4 Chain Integrity

Each receipt's `previous_hash` field creates a backward link to the preceding receipt. The `chain_index` field provides a zero-based position in the ledger. Together, these fields create a structure analogous to a blockchain: any modification to a historical receipt invalidates all subsequent hashes, making tampering detectable by any party with read access to the ledger.

---

8. External Memory

RIO uses Google Drive as a persistent external memory layer and ledger store. This design decision reflects the current implementation's architecture, where the system

operates across ephemeral AI agent sessions that do not retain state between interactions.

8.1 Folder Structure

The system maintains a structured folder hierarchy in Google Drive under a root folder called “One”:

Folder	Purpose
`/actions`	Records of every tool call and execution
`/concepts`	Distilled patterns and framework mappings
`/ledger`	Cryptographic receipts and audit logs
`/drafts`	Working papers and unrefined ideas
`/archive`	Compressed history to keep live context lean

8.2 System Index

A `core-spec-v1.json` file serves as the system’s source of truth, containing the current version, active protocols, authority declarations, and references to all governed components. This index is read at session initialization to restore system state.

8.3 Receipt Filing

When a governed action completes, the receipt is filed to the `/ledger` folder as a timestamped Markdown document. This creates a human-readable audit trail that is simultaneously machine-verifiable (via the embedded hashes) and accessible through standard file browsing.

8.4 Design Rationale

Using Google Drive as the persistence layer provides several properties relevant to the current implementation stage: the data is accessible across different AI agent platforms, it is backed by Google’s infrastructure for durability, it supports OAuth-

based access control, and it provides a familiar interface for human review. In a production deployment, the ledger would migrate to a dedicated append-only datastore, but the architectural pattern — external persistence with hash-chain integrity — remains the same.

---

9. Example Flow: Self-Proving Email

On March 29, 2026, the RIO system executed its first live governed action: sending an email that contained its own cryptographic receipt. This demonstration proves that the system can govern a real-world action end-to-end.

9.1 Sequence

1. **Intent Created.** The AI agent proposed sending an email from `riomethod5@gmail.com` to `riomethod5@gmail.com` with the subject "One: Self-Proving Demo Email — Receipt Embedded."
2. **Policy Evaluated.** The policy engine classified `send_email` as requiring human approval.
3. **Human Approved.** The human authority (Brian K. Rasmussen) reviewed the intent and approved execution via the Manus agent confirmation interface.
4. **Hashes Computed.**
   - Intent hash: `6231b1c7a4210be3631bdb80c2d431e049e7eeea169c28f2783f781222490f4a`
   - Action hash: `b7790d5ce64865234d65b383919b4cafab3de11640281d91112e5c3aa274949a`
   - Verification hash: `7cd08e5bb03afafc03420c145eef0f9446470bda4f3e91cb6c57e1a9d4ffa193`
5. **Email Sent.** The Gmail connector sent the email via the Gmail API. The email body contained the full receipt including all three hashes.
6. **Receipt Filed.** The receipt was written to Google Drive at `One/root/receipts/2026-03-29-demo.md`.

9.2 Verification

Any third party can verify this action by:

1. Taking the email content and computing SHA-256 of the canonicalized JSON.
2. Comparing the result to the `action_hash` in the receipt.
3. If they match, the email content is exactly what was authorized — no modification occurred between approval and delivery.

The email is simultaneously the product and the proof. It demonstrates that the governed execution pipeline works end-to-end with a real external system.

---

10. Security Model

RIO's security model is built on the principle that AI agents are untrusted requesters. The system does not rely on the agent's cooperation, alignment, or good behavior.

10.1 Fail-Closed Architecture

The default state of the execution gateway is **deny**. If any of the following conditions are true, execution is blocked:

- No approval record exists for the intent
- The approval signature is invalid or forged
- The approval has expired
- The approval has already been consumed
- The execution parameters do not match the original intent
- The ledger service is unavailable
- The signature verification service is unavailable

The system does not degrade gracefully into permissive mode. If governance cannot be enforced, no action proceeds.

10.2 Threat Model

The system has been tested against the following attack vectors, with all ten verification tests passing:

Test	Attack Vector	Result
V-001	Execution without approval	Blocked (HTTP 422)
V-002	Execution with valid approval	Success (HTTP 200)
V-003	Replay attack (reuse approval)	Blocked (HTTP 409)
V-004	Payload tampering after approval	Blocked (HTTP 409)
V-005	Approval revoked before execution	Blocked (auto-expire)
V-006	Direct executor call (bypass gateway)	Blocked (HTTP 403)
V-007	Invalid / forged signature	Blocked (ECDSA decode failure)
V-008	Ledger unavailable	Fail-closed (HTTP 503)
V-009	Approval service unavailable	Fail-closed (verification fails)
V-010	Duplicate execution	Blocked (HTTP 409)

10.3 Credential Isolation

The AI agent never holds OAuth tokens or API credentials directly. All credentials are stored at the execution gateway. The agent can only submit intents to the governance pipeline; it cannot invoke connectors or external APIs without passing through the gate.

10.4 Server-Side Enforcement

All authorization checks are performed server-side at the execution gateway. There is no client-side enforcement that could be bypassed by modifying the frontend, disabling JavaScript, or intercepting network requests.

---

11. Current Implementation

The following components exist as working code and have been verified through automated testing and live demonstration.

11.1 What Exists Today

Component	Implementation	Status
Execution Gateway	Replit-hosted Node.js server with ECDSA signature verification	Deployed, verified
Receipt Protocol	Five JSON schemas (ActionRequest, AIRecommendation, ApprovalRecord, ExecutionRecord, Receipt)	Published on GitHub
Ledger	Hash-chained receipt storage in Google Drive	Operational, one live receipt
Gmail Connector	Live email execution via Gmail MCP	Connected, verified
Google Drive Connector	File and folder operations via `gws` CLI	Connected, verified
GitHub Connector	Issue, PR, and commit operations via `gh` CLI	Connected, verified
Google Calendar Connector	Event operations via `gws` CLI	Simulated (scope limitation)
Verification Test Suite	10 automated security tests	$^{10}/_{10}$ passing
Demo Site	Public-facing site at riodemo-ux2sxdqo.manus.space	Live

11.2 Repository Structure

```
rio-system/
├── frontend/           # Demo site UI
├── backend/            # Gateway server and API routes
├── crypto/             # Signature generation and verification
├── ledger/             # Ledger read/write operations
├── database/           # Schema and persistence
├── connectors/         # Gmail, Drive, GitHub, Calendar modules
│   ├── base.ts         # RioConnector interface definition
│   ├── gmail.ts        # Gmail connector implementation
│   ├── google-drive.ts # Drive connector implementation
│   ├── github.ts       # GitHub connector implementation
│   └── registry.ts     # Connector registration
├── rio-receipt-protocol/
│   ├── schemas/        # JSON Schema definitions
│   ├── docs/           # Protocol and ledger specifications
│   └── examples/       # Sample receipts and chains
├── tests/              # Verification test suite
└── verification_logs/  # Test results
```

---

12. Roadmap

The following components are planned for future development. Each extends the existing architecture without modifying the core invariants.

12.1 Policy Engine (Learning Loop)

The current implementation uses static policy rules. The planned Policy Engine introduces a Learning Loop that analyzes the history of receipts to suggest policy refinements. For example: “You have approved all emails to alice@example.com under 200 words in the last 30 days. Should this become an AUTO_APPROVE rule?” Policy suggestions require human approval before activation — the Learning Loop cannot bypass governance.

12.2 Azure Deployment

The current gateway is hosted on Replit. A planned migration to Azure provides enterprise-grade infrastructure, integration with Azure Active Directory for organizational identity management, and compliance with enterprise security requirements.

12.3 Permanent Runtime

The current system operates within ephemeral AI agent sessions. A permanent runtime would maintain continuous state, process incoming intents asynchronously, and provide a persistent API endpoint that any AI agent platform can integrate with.

12.4 Multi-Agent Governance

As AI systems increasingly delegate tasks to other AI agents, RIO's governance model extends to agent-to-agent interactions. When Agent A asks Agent B to perform an action, the request passes through RIO's governance pipeline with the same authorization requirements as a human-initiated request.

---

13. Use Cases

RIO applies to any scenario where an AI agent performs actions with real-world consequences. The following examples illustrate the governance pattern across different domains.

13.1 Governed Email

An AI assistant drafts an email on behalf of a user. RIO intercepts the send action, presents the draft to the user for approval, and only sends the email after receiving a signed authorization. The receipt is embedded in the email itself, proving to the recipient that the message was human-authorized.

13.2 Financial Operations

A finance agent identifies an outstanding invoice and proposes payment. RIO's policy engine evaluates the amount against configured thresholds. Payments under

$100 areauto - approved; paymentsover$1,000 require manager-level authorization. The receipt records the exact amount, the approver's identity, and the timestamp.

13.3 Infrastructure Management

A DevOps agent proposes a production deployment. RIO requires director-level approval for production changes. The receipt includes the commit hash, the deployment target, and the approver's identity, creating an auditable deployment history.

13.4 Data Operations

An agent tasked with GDPR compliance receives a data deletion request. RIO ensures the deletion target matches the requesting user's data, requires compliance officer approval, and generates a receipt proving the deletion was authorized and executed correctly.

13.5 Subscription and Service Management

An AI assistant identifies unused subscriptions and proposes cancellation. RIO presents each cancellation to the user for approval, executes the cancellation through the appropriate connector, and files a receipt documenting what was cancelled, when, and by whose authority.

---

14. Definitions

Term	Definition
RIO	Runtime Intelligence Orchestration. A fail-closed governance runtime for AI agent execution.
Intent	A structured, immutable record of a proposed action submitted by an AI agent.
Policy Engine	The component that evaluates intents against human-authored rules and returns a governance verdict.
Gate / Gate UI	The human-facing interface where authorities review and approve or deny pending actions.
Execution Gateway	The server-side enforcement point that verifies approvals and dispatches actions to connectors.
Connector	An execution module that interfaces with an external system (Gmail, Drive, GitHub, etc.).
Receipt	A cryptographically signed record binding together the intent, approval, execution, and outcome of a governed action.
Ledger	An append-only, hash-chained audit log of all receipts.
Fail-Closed	A design principle where the default state is deny; execution requires explicit authorization.
Intent Hash	SHA-256 hash of the canonicalized intent JSON. Proves the request was not altered.
Action Hash	SHA-256 hash of the canonicalized execution parameters. Proves the action matches what was authorized.
Verification Hash	SHA-256 hash of the canonicalized execution result. Proves the outcome was recorded accurately.
Nonce Registry	A single-use token registry that prevents replay attacks by ensuring each approval is consumed exactly once.

Term	Definition
Three-Loop Architecture	RIO's structural design: Intake (goal → intent), Governance (policy → approval → execution → verification), Learning (ledger → policy improvement).
AUTO_APPROVE	Policy verdict allowing execution without human intervention for low-risk actions.
REQUIRE_HUMAN	Policy verdict blocking execution until a human authority approves.
AUTO_DENY	Policy verdict permanently blocking execution for policy-violating actions.
Confidence	A percentage-based classification on AI recommendations indicating the system's certainty, with automatic escalation below configurable thresholds.

---

15. Conclusion

RIO introduces governed execution as a new architectural layer for AI systems. It is not an alternative to model alignment, prompt engineering, or safety training — it is a complementary structural layer that enforces governance at runtime regardless of the AI model's internal state.

The core insight is that AI agents should be treated as untrusted requesters in the same way that web applications treat user input as untrusted. The solution is the same in both cases: validate everything server-side, enforce authorization structurally, and maintain an immutable audit trail.

RIO's contribution is a concrete implementation of this principle: a deterministic pipeline (Intent → Governance → Execution → Receipt), four non-negotiable invariants (Identity, Consent, Persistence, Confidence), a cryptographic receipt protocol with hash-chained ledger integrity, and a working reference implementation with verified security properties.

The system has been demonstrated with live connectors (Gmail, Google Drive, GitHub), verified against ten attack vectors with a $^{10}/_{10}$ pass rate, and documented with formal JSON schemas for every protocol artifact. The architecture is connector-agnostic: any

external system accessible via API can be governed by adding a connector that implements the standard interface.

Governed execution does not slow AI systems down. It makes them accountable. Every action is authorized, executed, verified, recorded, and available for audit. The system enforces the rules — not the AI.

References

1. RIO Protocol White Paper v2.0.0. Rasmussen, B.K. March 2026. Google Drive.
2. RIO Governance Runtime Specification v1.1. Rasmussen, B.K. March 2026. Google Drive.
3. RIO System Architecture — Six-Primitive Kernel. Rasmussen, B.K. 2026. Google Drive.
4. RIO Receipt Protocol v0.1. github.com/bkr1297-RIO/rio-system/rio-receipt-protocol.
5. RIO System Overview. github.com/bkr1297-RIO/rio-system/RIO_SYSTEM_OVERVIEW.md.
6. RIO Connector Architecture. github.com/bkr1297-RIO/rio-system/connectors/README.md.
7. RIO Verification Results — $^{10}/_{10}$ PASS. github.com/bkr1297-RIO/rio-system/DEMO_WALKTHROUGH.md.
8. The Structural Read & The Dependency Trail. Rasmussen, B.K. March 2026. Google Drive.
9. RIO Sovereign Witness Statement. March 29, 2026. Google Drive.